

Array $\rightarrow$ 1D

An array is a collection of elements of the same type, stored in contiguous memory locations. We can access each element using its index.

```
int[] arr = {1, 5, 20, 10, 7, 8};
```

`arr[0]` $\rightarrow$ first element (1)

`arr[4]` $\rightarrow$ 8

`arr[8]` $\rightarrow$ `ArrayIndexOutOfBoundsException`

Matrix (2D Array)

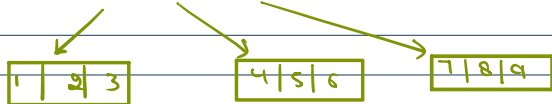
It is a data structure that stores elements in a grid-like format, consisting of rows and columns.

A matrix is a 2D array - data arranged in rows and columns. It is essentially an array of arrays, where each element of the outer array is itself an array.

`arr` $\rightarrow$

3	2	4
---	---	---

 $\rightarrow$ {1D} $\rightarrow$ one dimensional array

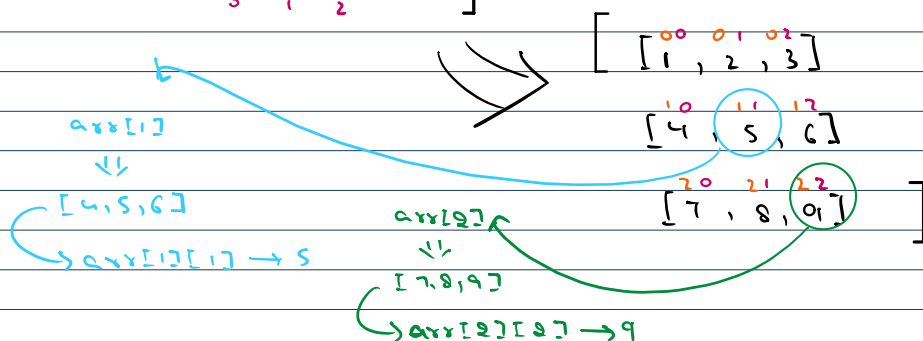


$\begin{bmatrix} \text{0} & \text{1} & \text{2} \\ [1, 2, 3] \end{bmatrix}$

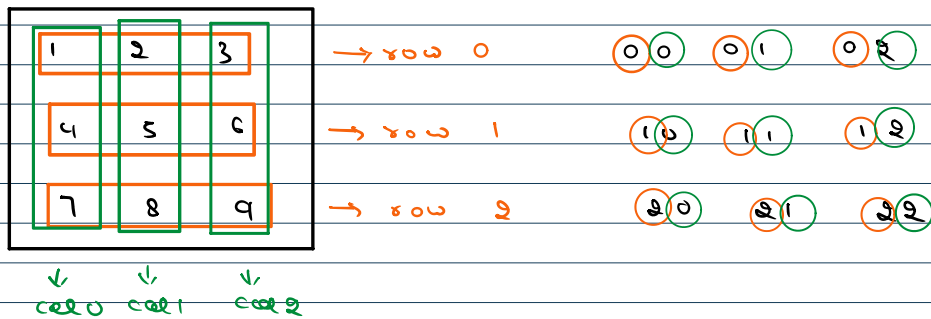
$\begin{bmatrix} \text{0} & \text{1} & \text{2} \\ [4, 5, 6] \end{bmatrix}$

$\begin{bmatrix} \text{0} & \text{1} & \text{2} \\ [7, 8, 9] \end{bmatrix}$

$\rightarrow$ {2D} $\rightarrow$ two dimensional array



Rows $\rightarrow$ horizontal line of elements
Columns $\rightarrow$ vertical line of elements



Dimension :- rows X col x (size)
3 X 3

Indexing → ?

matrix[0][2] → row 0, col 2 → 3

1D → int[] arr = new int[3];

2D → int[][] arr = new int[3][4];

Annotations for 2D array declaration:

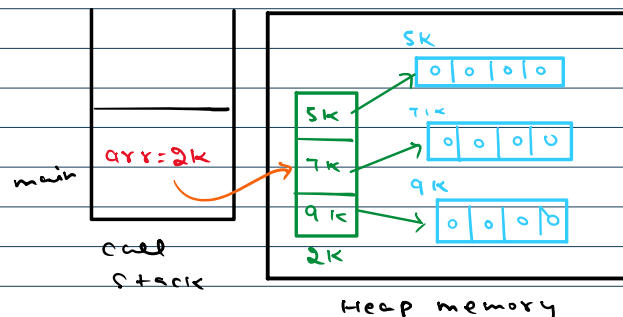
- 10 → int: data type
- arr: name of the array
- new: keyword
- int[3][4]: array size (rows = 3, cols = 4)
- arr: reference variable of array which stores the address of the object in heap memory

not

2D arrays are stored in contiguous memory location.

```
public class ArraysDemo {
    public static void main(String[] args) {
        int[][] arr = new int[3][4];

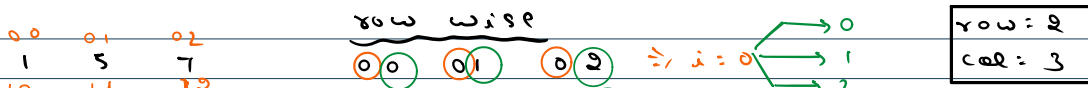
        System.out.println(arr);
        System.out.println("Hello Akarsh!");
    }
}
```



int[][] arr = new int[n][m]

Total no. of 1D arrays formed ? (n+1)

Print the array



0 0 0
1 5 7
2 9 1

row wise

0 0 0 1 0 2
1 0 1 1 2

$\Rightarrow i = 0$
 $\Rightarrow i = 1$

row = 2
col = 3

for (int i = 0; i < row; i++) {

for (int j = 0; j < col; j++) {
 SOP(arr[i][j]);

}

col wise

1 2
5 9
7 1

0 0 1 0
0 1 1 1
0 2 1 2

$i = 0$
 $i = 1$
 $i = 2$

for (int i = 0; i < col; i++) {

for (int j = 0; j < row; j++) {

SOP(arr[j][i]);

}

}

item = 14

0 1 2 3 4 $\rightarrow$ col

0	1	4	7	11	15
1	2	5	8	12	19
2	3	6	9	16	22
3	10	13	14	17	24
4	18	21	23	26	30

row = 0 { 0 }
col = 4 { m-1 }

while (

if (arr[row][col] == item) {

return true;

}

else if (arr[row][col] > item) {

col--;

}

else if (arr[row][col] < item) {

row++;

}

}

return false

item = 20

0 1 2 3 4 $\rightarrow$ col

0	1	4	7	11	15
1	2	5	8	12	19
2	3	6	9	16	22
3	10	13	14	17	24
4	18	21	23	26	30

$\Rightarrow$

Loop exit condition
 $\Rightarrow$
row < arr.length
&&
col >= 0

Loop will work in this scenario

row $\Rightarrow$ row = 5

class Solution {

public boolean searchMatrix(int[][] matrix, int target) {

```
int rows = matrix.length;
int cols = matrix[0].length;
```

```
for(int i = 0; i < rows; i++) {
    if(binarySearch(matrix[i], target) != -1) {
        return true;
    }
}
return false;
}
```

```
public static int binarySearch(int[] arr, int item) {
    int n = arr.length;
    int low = 0;
    int high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == item) {
            return mid;
        } else if (arr[mid] > item) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return -1;
}
```

```
}
```

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int rows = matrix.length;
        int cols = matrix[0].length;

        for(int i = 0; i < rows; i++) {
            if(binarySearch(matrix[i], target) != -1) {
                return true;
            }
        }
        return false;
    }
}
```

```
public static int binarySearch(int[] arr, int item) {
    int n = arr.length;
    int low = 0;
    int high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == item) {
            return mid;
        } else if (arr[mid] > item) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
}
```

```

        return -1;
    }
}

class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int row = 0;
        int col = matrix[0].length - 1;

        while(row < matrix.length && col >= 0) {
            if(matrix[row][col] == target) {
                return true;
            } else if(matrix[row][col] > target) {
                col--;
            } else {
                row++;
            }
        }
        return false;
    }
}

```

Jagged Array

where rows can have different lengths.

```

arr = {
    { 1, 2 },
    { 3, 4, 5 },
    { 1 }
}

```

```

int[][] jagged = new int[3][4]; ✓
int[][] arr = new int[3][4]; ✗

```